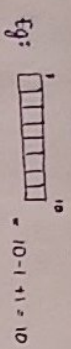


⇒ Data Type vs Data Structure
 - Type of data - kind of organizing the data

⇒ Linear vs Non Linear Data Structure
 - one element is connected to multiple elements
 - one element is connected to multiple elements

⇒ Array

→ One dimensional arrays
 Length of array = $UB - LB + 1$



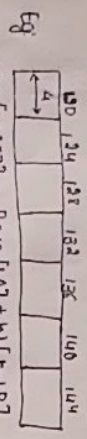
→ Representation of linear arrays in memory

(Address of element $LA[k]$)
 $LOC[LA[k]] = Base(LA) + W(k - LB)$

W = no of words per memory cell

LA = name of array

Base (LA) → address of first element



→ Traversing linear Array

(to visit every element exactly once)

LA is a linear array with lower bound LB & upper bound UB . The algorithm traversing array applying an equation process to each element of LA .

1. [Initialize counter] $at\ k = LB$
2. Repeat step 3 and 4 while $k \leq UB$
3. [Visit element] Apply process to $LA[k]$
4. [Increase counter] $at\ k = k + 1$
5. Exit

→ Inversion in a linear array
 This algorithm inverts an element into a kth position in LA

1. [Initialize counter] $at\ j = N$
2. Repeat step 3 and 4 while $j \geq k$
3. [Move j th element downward]
 $at\ LA[j+1] = LA[j]$

4. [Decrease counter] $Set\ j = j - 1$

[end of step 3 loop]

5. [Insert element] $at\ LA[k] = item$

6. [Print N] $set\ N = N - 1$

7. Exit

→ Deletion in linear array
 Delete ($LA, N, k, item$)

1. $at\ item = LA[k]$

2. Repeat for $j = k$ to $N - 1$

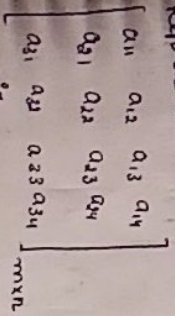
3. [Move $j + 1$ element upward]
 $at\ LA[j] = LA[j + 1]$
 [end of loop]

3. [Print the no of elements in LA]
 $at\ N = N - 1$

4. Exit

⇒ Multidimensional Arrays

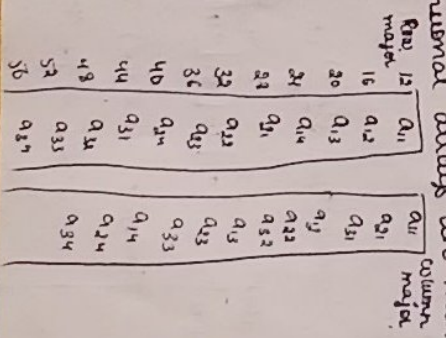
→ Representation of 2 dimensional array in memory



$LOC = Base(A) + W[N(2 - 1) + (k - 1)]$

$= 12 + 4(9) = 12 + 36 = 48$

(if element $[3][2]$)



→ Col^m major $a[2][3]$ dimension mth

$$LOC[a][b][c] = Base(a) + m[b - (k-1)] + (c-1)$$

$$= 12 + 4(2) = 40$$

→ General Multidimensional Array

The length L_i of dimension i of c is the no of elements in the index $L_i = UB - LB + 1$

For a given subscript K_i , the effective index E_i of L_i is no of indices preceding K_i in the index list and can be calculated as $E_i = K_i - LB$.

→ Col^m major

$$LOC[c][k_1, k_2, \dots, k_n] = Base(c) + m[(E_1 \dots E_{n-1} L_n - 2) + \dots + E_2] L_n E_3 h + E_1$$

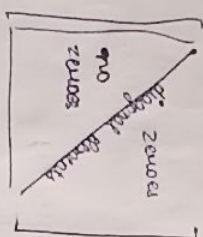
→ Row major

$$LOC[l_1, l_2, \dots, l_n] = Base(c) + m[(E_1 E_2 + E_3) L_2 L_3 L_4 + \dots + E_2] L_n + E_1$$

→ Sparse matrices

number of zeros > no of non-zero elements

8 Assume are inefficient to store elements of sparse matrices as compared to linked lists.



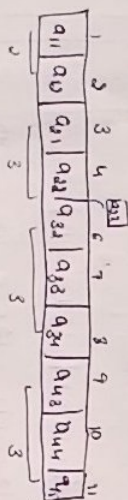
To find a_{jk} :

Elements with $(j-1)$ are completely stored,

$$= (j-1)(j-1+1) \quad (\text{using } n(n+1)/2)$$

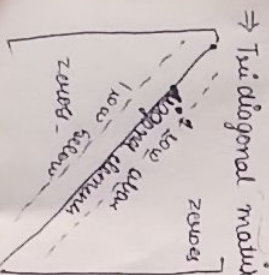
$$= \frac{j(j-1)}{2} \quad \left[\begin{array}{l} \text{For } a_{42}, = \frac{4(4-1)}{2} \\ = \frac{4(3)}{2} = 6 \end{array} \right]$$

$$(position) a_{jk} = \frac{j(j-1)}{2} + k \quad [6 + 2 = 8]$$



For a_{jk} : $2 + 3(j-2)$ - already stored completely,

$$= [2 + 3(j-2)] + (k-j+2)$$



8 Decide on efficient way of storing a symmetric matrix A in memory

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix}$$

$$A = A^T, a_{ij} = a_{ji}$$

$$A[j:k] = \frac{j(j+1)}{2} \quad \text{if } j > k$$

$$A[j:k] = \frac{j(k+1)}{2} \quad \text{if } j < k$$

$$A[j:k] = k(k+1)/2 + j \quad \text{if } j = k$$

23/8/25

→ Algorithm Complexity and Asymptotic notation

M-Algorithm

Two measures of efficiency

Time

Space

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < 2^n < 3^n < \dots$$

O Big-Oh upper bound

Ω Big omega lower bound

Θ theta Average bound

→ Big-Oh

The fn. $f(n) = O(g(n))$ if $\exists k$ constant c and n_0 such that $f(n) \leq (n g(n)) \forall n \geq n_0$

$$\text{Eg: } f(n) = 2n+3 \quad \forall n \geq n_0$$

$$2n+3 \leq c \cdot g(n) \quad n \geq 1 \quad O(n)$$

→ Omega notation

The fn. $f(n) = \Omega(g(n))$ if $\exists k$, constant c and n_0 such that $f(n) > g(n) \quad \forall n \geq n_0$

$$f(n) > g(n) \quad \forall n \geq n_0$$

$$\text{Eg } f(n) = 2n+3 \quad \forall n \geq n_0$$

$$2n+3 \geq 2n \quad n \geq 1 \quad \Omega(2n)$$

$$2n+3 \geq 2 \log n \quad \Omega(\log n)$$

→ Theta notation

The fn. $f(n) = \Theta(g(n))$ if $\exists k, c$, constant c_1, c_2 and n_0 such that

$$c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

Eg

$$f(n) = 2n+3$$

$$c_1 \times g(n) \leq 2n+3 \leq c_2 \times g(n)$$

$$2 \times n \leq 2n+3 \leq 4 \times n \quad \Theta(n)$$

Unit-II SEARCHING and SORTING

⇒ Insertion (A, N)

1. Set $A[0] = -\infty$
2. Repeat steps 3 to 5 for $k=2, 3, \dots, N$
3. Let $\text{temp} = A[k]$ and $\text{ptr} = k-1$

4. Repeat while $\text{temp} < A[\text{ptr}]$

a. Set $A[\text{ptr}+1] = A[\text{ptr}]$

b. Set $\text{ptr} = \text{ptr} - 1$

[end loop]

5. Set $A[\text{ptr}+1] = \text{temp}$

6. Return.

⇒ Merge sort

Merging $[A, R, B, S, C]$

1. [Initialize] Set $NA=1, NB=1, PTR=1$

2. [compare] Repeat while $NA \leq R$ and $NB \leq S$

If $A[NA] < B[NB]$, then

a. [Assign element from A to C]

Set $C[PTR] = A[NA]$

b. Let $\text{ptr} = \text{ptr} + 1, NA = NA + 1$

else

a. [Assign element from B to C]

Set $C[PTR] = B[NB]$

b. Set $\text{ptr} = \text{ptr} + 1, NB = NB + 1$

end of if

end of loop

3. [Assign remaining element to C]

If $NA > R$, then

Repeat for $k=0, 1, 2, \dots, S-NB$

Set $C[\text{ptr}+k] = B[NB+k]$

for a loop

else

Repeat for $k=0, 1, 2, \dots, R-NA$

Set $C[\text{ptr}+k] = A[NA+k]$

[end of loop]

4. Exit

→ Merge Sort

The N element array A is composed of sorted sub-arrays, where, each sub-array has elements except possibly the last sub-array which may have fewer than L elements. The procedure merges pairs of sub-arrays of A & assigns them to the array B.

1. Set $Q = \text{INT}(N/2 \times L), S = 2 \times L \times Q$

$R = N - S$

2. Repeat for $J=1, 2, \dots, Q$

a. Set $LB = 1 + (2J-2) \times L$

b. Call Merge ($A, LB, A, LB, LB+L, B, LB$)

3. [Only one sub-array left?]

If $R < L$, then;

Repeat for $J=1, 2, \dots, R$

Set $B[S+J] = A[S+J]$

[end of loop]

else call Merge ($A, L, S+1, A, R, L+S+1, B, S+1$)

[end of if else]

4. Return.

→ Merge sort (A, N)

1. Set $L=1$

2. Repeat step 3 to 6 while $L < N$

3. Call Merge pass (A, N, L, B)

4. Call Merge pass ($B, N, 2 \times L, A$)

5. Set $L = 4 \times L$

6. Exit

→ Merge ($A, R, LBA, S, LBB, C, LBC$)

1. Set $NA = LBA, NB = LBB, PTR = LBC, UBA = LBA+R-1, UBB = LBB+5-1$

2. Same as merging algorithm except R is replaced by UBA and

5 by UBB

3. " " " "

4. Return.

⇒ Radix Sort

348, 143, 361, 423, 538, 129, 221, 543, 366

For one's place

0	1	2	3	4	5	6	7	8	9
	361	143	423	348	538	366		129	
	321	423	343						
		321	538	143	361				
		423	348		366				
		129							

⇒ 361, 321, 143, 423, 348, 866,

348, 538, 129

For ten's place

⇒ 321, 423, 129, 538, 143, 348, 348,

361, 366

For hundred's place

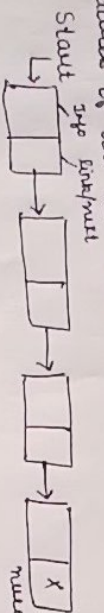
⇒ 129, 143, 321, 348, 361, 366, 423,

Unit-III

21/8/23

⇒ Linked List

Linked list or one way list is a linear collection of data elements called nodes where linear order is given by means of pointers, each node is divided into 2 parts, first part contains the info of element & second part called the link/next pointer field contains the address of the next node in the list.



The pointer of last node contains a special value called null pointer which is a way invalid address. Linked list also contains a list pointer variable called start which contains address of first node in the list

⇒ Representation of linked list in memory

⇒ Traversing a linked list

1. Set ptr = start
2. Repeat step 3 and 4 while ptr ≠ null
3. Apply process to info [ptr]
4. Set ptr = link [ptr] [ptr now points to next node]
[end of step 2 loop]
5. Exit

⇒ Insert algorithm to insert the elements of the linked list.

- Change 3. to print info [ptr] in the above procedure

⇒ Write an algorithm to count number of elements in a linked list.

1. Set ptr = start, num = 0
2. Repeat step 3 and 4 while ptr ≠ null
num = num + 1
- 3.
4. Set ptr = link [ptr]
[end of step 2 loop]
5. Exit

⇒ Insertion after a given node

INS LOC [Info, link, start, LOC, item]

This algo inserts item so that item follows the node with location LOC or inserts item as the first node when LOC = null

1. Create a new node
2. Set info [new] = item
3. If LOC = null, then [Insert as first node]
Set link [new] = start and start = new
4. Insert after node with location LOC
Set link [new] = link [LOC] and link [LOC] = new
[End of if else]
5. Exit.

11/9/25

⇒ Inserting into a sorted linked list

This procedure finds the location LOC of the last node in a sorted linked list such that $info\ of\ LOC < item$ or $LOC = null$

Find A[info, link, start, item, loc]

1. [List empty] if $start = null$, then

set $loc = null$ and Return

2. [Special case] If $item < info[start]$,

then set $loc = null$ and return

3. set $save = start$ and $ptr = link[start]$ [critical point]

4. Repeat step 5 & 6 while $ptr \neq null$

5. If $item < info[ptr]$, then

set $loc = save$ and return

[end of if ptr]

6. Set $save = ptr$ and $ptr = link[ptr]$

[end of step 4 loop]

7. Set $loc = save$

8. Return

⇒ Insert [info, link, start, item]

1. Call find [info, link, start, item, loc]

2. Call insert [info, link, start, loc, item]

3. Exit

⇒ Insertion at end

1. Create a new node

2. Set $info[new] = item$ set $ptr = start$

3. Repeat while $link[ptr] \neq null$

$ptr = link[ptr]$

4. $link[new] = null$

5. $link[ptr] = new$

6. Exit

⇒ Deletion from linked list

→ Deletion at beginning

1. Set $item = info[start]$

2. set $ptr = start$

3. set $start = link[start]$

4. Free ptr

→ Deletion at end

1. Set $ptr = start$, $save = null$

2. Repeat while $link[ptr] \neq null$

3. set $save = ptr$

4. set $ptr = link[ptr]$

5. If $save = null$,

ptr = start

about = null

else

$link[save] = null$

[end of if ptr]

6. Exit

→ Deleting node given address

Del [info, link, start, loc, locp]

1. If $locp = null$, then

set $start = link[start]$ [delete first node]

else

set $link[locp] = link[loc]$

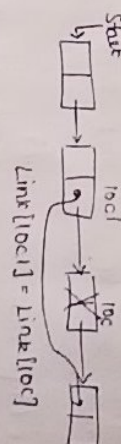
[delete node n]

[end of if ptr]

2. Free loc

3. Exit

This algorithm deletes the node n with location loc, locp is location of the node which precedes n or when no prev node, locp = null



→ Deleting a node with given item's information:

Find B (info, link, start, item, LOC, locp)

1. [list empty?] If start = null, then

start loc = null and locp = null and

return

[end of list]

2. [item in given node] If info [start] = item, then

start loc = start and locp = null and return

[end of list]

3. Set save = start and ptr = link [start]

4. Repeat step 3 and 4 while ptr ≠ null

5. If info [ptr] = item, then

start loc = ptr and locp = save and return

[end of list]

6. start save = ptr and ptr = link [ptr]

[end of step 4 loop]

7. start loc = null

8. return

This procedure finds the location loc of the first node in which certain item and location locp of node preceding it. If item does not appear in the list then procedure sets loc = null and if item appears in a node, then it sets locp = null.

15/9/95

Delete (info, link, start, item)

1. Call Find B (info, link, start, item, LOC, locp)

2. If LOC = null, then write item not in list and exit.

3. [Delete node]

If locp = null, then

start = link [start]

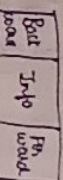
[delete given node]

else

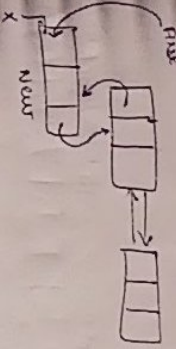
start link [locp] = link [loc]

end of if else

→ Doubly linked list [Two way list]

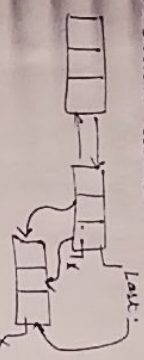


→ Insertion at beginning



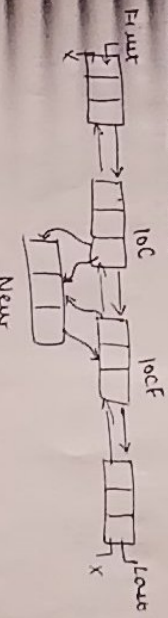
Forward [New] = First
Back [First] = New
Back [New] = null
First = New

→ Insertion at end



Forward [Last] = New
Back [New] = Last
Forward [New] = null
Last = New

→ Insertion at middle



1. locf = Forward [loc]
2. Forward [New] = locf
3. Back [locf] = New
4. Back [New] = loc
5. Forward [loc] = New

If locp = null

// Insertion at first node

else if Forward [locp] = null

// Insertion at last node

else

insertion in middle.

→ Deletion in a doubly linked list



Deletion at beginning

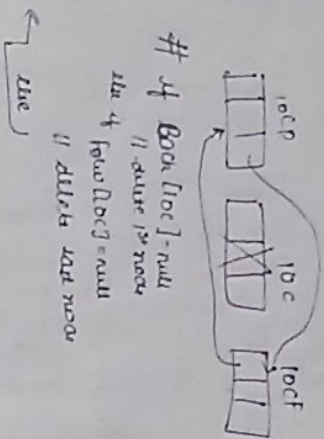
- ① $LOC F = \text{front} [\text{front}]$
- ② $\text{Back} [\text{LOC F}] = \text{null}$
- ③ $\text{front} = \text{LOC F}$

⇒ Deletion at end

- ① $\text{LOC D} = \text{Back} [\text{last}]$
- ② $\text{front} [\text{LOC D}] = \text{null}$
- ③ $\text{last} = \text{LOC F}$

⇒ Insert Deletion in middle

- ① $\text{locb} = \text{Back} [\text{LOC}]$
- ② $\text{loc F} = \text{front} [\text{LOC}]$
- ③ $\text{front} [\text{locb}] = \text{loc F}$
- ④ $\text{Back} [\text{loc F}] = \text{locb}$



⇒ Application of linked list
→ Linked list are used to store polynomials

• Addition of two polynomials:

1. Set $\text{ptr1} = \text{store1}$
 $\text{store2} = \text{store2}$

2. Repeat while $\text{ptr1} \neq \text{null}$ and $\text{ptr2} \neq \text{null}$

3. Create a new node

4. If $\text{deg} [\text{ptr1}] > \text{deg} [\text{ptr2}]$

$\text{deg} [\text{new}] = \text{deg} [\text{ptr1}]$

$\text{coef} [\text{new}] = \text{coef} [\text{ptr1}] + \text{coef} [\text{ptr2}]$

$\text{ptr1} = \text{link} [\text{ptr1}]$

$\text{ptr2} = \text{link} [\text{ptr2}]$

else if $\text{deg} [\text{ptr1}] < \text{deg} [\text{ptr2}]$

$\text{deg} [\text{new}] = \text{deg} [\text{ptr2}]$

$\text{coef} [\text{new}] = \text{coef} [\text{ptr2}]$

$\text{ptr1} = \text{link} [\text{ptr1}]$

else

$\text{deg} [\text{new}] = \text{deg} [\text{ptr2}]$

$\text{coef} [\text{new}] = \text{coef} [\text{ptr2}]$

$\text{ptr2} = \text{link} [\text{ptr2}]$

[end of 4th step]

5. If $\text{store} = \text{null}$

$\text{store} = \text{new}$

$\text{link} [\text{new}] = \text{null}$

$\text{last} = \text{new}$

else

$\text{link} [\text{last}] = \text{new}$

$\text{link} [\text{new}] = \text{null}$

$\text{last} = \text{new}$

[end of 4th step]

else

Repeat while $\text{ptr1} \neq \text{null}$

a. Create a new node

b. $\text{deg} [\text{new}] = \text{deg} [\text{ptr1}]$

c. $\text{coef} [\text{new}] = \text{coef} [\text{ptr1}]$

d. Increase steps

[end of while loop]

7. Exit

Unit - IV STACKS...

⇒ LIFO [Last In First Out]

⇒ Operations

→ Insertion [Push] → Deletion [Pop]

→ Traversing

Stack is a list of elements in which an element may be inserted or deleted only at one end called the top of the stack. This means that elements are removed from stack in reverse order of that in which they were inserted into the stack.

→ Push is the term used to insert an element into stack.

→ Pop is term used to delete an element from stack.

⇒ Representation of Stack in memory

⇒ Array representation.

Push[stack, Top, MaxSize, item]

1. If Top = maxSize: Then [Stack already full]

Print overflow and return

2. Set Top = Top + 1 [Increased top by 1]

3. Set stack[Top] = item

4. Return

Pop[stack, Top, maxstk, item]

1. [Stack has an item to be removed]

If Top = 0, then print underflow and return

2. Set item = stack[Top]

3. Set Top = Top - 1

4. Return

⇒ Linked representation

Push: [insert algo of insertion at beginning, replace stack by top]

Pop: [delete algo of deletion at beginning, replace " " " "]

⇒ Applications of Stack

⇒ Parenthesis balancing

⇒ Evaluation of postfix expression

infix: a+b
postfix: a+b (Polish notation)
postfix: ab+ (Reverse " ")

1. All right parenthesis, at the end of P, thus act as a sentinel

2. Scan from left to right & repeat steps 3-4 for each element of P until sentinel is encountered.

3. If an operand is encountered, push it onto stack

4. If an operator is encountered
• Remove two top elements of stack

A is low element & B is next to top

① Evaluate EXA

• Push operand onto stack

[End of ex]

5. Set value = Top element of stack.

18/9/25

⇒ Conversion of infix to postfix expression

Suppose A is an arithmetic exp written in infix. This algo finds its equivalent postfix exp. P

1. Push '(' and add ')' at end of B

2. Scan B from left to right and repeat steps 3-6 for each element of A until the stack is empty

3. If an operand is encountered, add it to P

4. If a left parenthesis is encountered, push it onto stack

5. If an operator is encountered, a. repeatedly pop from stack & add it to P, each operator which has same precedence or higher precedence than the operator.

b. Add operator to stack

6. If right parenthesis is encountered, a. repeatedly pop from stack, & add it to P each operator until left parenthesis is encountered.

b. Remove ')' & do not add it to P.

[End of A ex]

[End of step 6]

7. End

Symbol

operand

stack

Equation

P

A

+

C

B

X

C

(

)

+

+

+

-	(+ C-	ABC*	)	(+	ABC* DEF 1/g *	-
(	(+ C-C	ABC*	*)	(+ x	"	
D	C+C-C	ABC*D	H	(+ *	ABC* DEF ↑/g *-H	
/	(+C-C/	ABC*D	)	-	ABC* DEF ↑/g *-H *	
E	(+C-C/	ABC*DE				
↑	C+C-C/↑	ABC* DE				
F	C+C-C/↑	ABC* DEF				
)	C+C-	ABC* DEF 1				
*	C+C-*	ABC* DEF 1/				
g	(↑) (-*	ABC* DEF 1/g				

$$Q1. (A \uparrow B \uparrow D) / ((E - F) + G)$$

Q2 12, 7, 3, -, 1, 2, 1, 5, +, *, +

Symmet	Matrix	P
A	(	A
↑	(↑	A
B	(↑	AB
+	(↑+	AB
D	(↑+	ABD
)	—	ABD↑
/	/	ABD↑↑
(	/(	ABD↑↑
E	/(	ABD↑↑E
E-	/(-	ABD↑↑E
F	/(-	ABD↑↑EF
)	/	ABD↑↑EF-
+	/+	ABD↑↑EF-
G	/+	ABD↑↑EF-G

$ABD + \uparrow EF - G + /$

20	4	3
----	---	---

20	1	-	-	5
----	---	---	---	---

20	5	-	2	0
----	---	---	---	--------------

20	5	-	20
----	---	---	----

20	5	3
----	---	---

$\Rightarrow$ auch ADT

Quick [A, N, Beg, End, Loc]

Then A is array of N elements, Parameters Beg and End contain the boundary values of the subset of A in which this procedure applies. LOC keeps track of position of the first element A[Beg] of the sub-set, during the procedure, local variable I might well contain the boundary values of the list of elements that have not been scanned.

1. [Initialize] set left = Beg.

Right = End and LDC = Beg

a [Scan from right to left]

a. Repeat while $A[\text{loc}] \leq A[\text{right}]$ and $\text{loc} \neq \text{right}$

$$\text{Right} = \text{Right} - 1$$

b. If $LOC = \text{ought}$ then Return

c. $\exists b \ A[b] \wedge [b] > A[\text{right}]$, then

(j) [Interchange $A[L0C]$ and $A[Right]$]

$$\text{temp} = A[\text{loc}], A[\text{loc}] = A[\text{right}]$$
$$A[\text{Right}] = \text{Temp}$$

(ii) At LOC = Right

Step 3

[end of str]

3. [Scan from left to right]

a. Repeat while $A[\text{right}] \leq A[\text{loc}]$ and $\text{right} \neq \text{loc}$

$$\text{left} = \text{left} + 1$$

[end of loop]

b. If $100 = \text{left}$, then return

c, If $A[\text{right}] > A[\text{loc}]$, then

ii) Interchange $A[\text{left}]$ and $A[\text{loc}]$

$$T_{emp} = A [Lbc], A [Loc] - A [Rgt]$$

$A[\text{upper}] = \text{temp}$

(ii) $\text{at } \text{LOC} = \text{left}$

(iii) go to step 2

[end of file]

Quick Sort

1. [Initialize] Top = Null
2. [Push boundary values of A onto stack when A has 2 or more elements]
 - If $N > 1$, then Top = Top + 1
 - Lower[1] = 1, Upper[1] = N
3. Repeat step 4 to 7 while Top $\neq$ Null
4. [Pop element from stack]
 - Let Beg = Lower[Top]
 - End = Upper[Top]
 - Top = Top - 1
5. Call Quick [A, N, Beg, End, LOC]
6. [Push left subarray onto stack when it has 2 or more elements]
 - If Beg < LOC - 1, then
 - Top = Top + 1, Lower[Top] = Beg
 - Upper[Top] = LOC - 1
 - [End of 4th step]
7. [Push right subarray on to stack ---]
 - If LOC + 1 < end, then
 - Top = Top + 1, Lower[Top] = LOC + 1
 - Upper[Top] = End
 - [End of 4th step]
8. Exit

Recursion

- $\Rightarrow$ Tower of Hanoi
- Tower (N, Beg, Aux, End)
1. If $N = 1$, then
 - a. Write Beg $\rightarrow$ end
 - b. return
 - [End of 1st step]
 2. [Move $n-1$ disks from Beg to Aux]
 - call Tower [N-1, Beg, end, Aux]
 3. Write Beg $\rightarrow$ end
4. [Move $n-1$ disks from Aux to Beg]
5. Return

Implementation of recursive procedures by stack

The illustration of recursive procedure P into non-recursive procedure works as follows:

1. List of active elements a stack STPAR for each variable parameter of stack STPAR for every local variable var.

2. When local variable var is a stack STADD to hold return addresses.

Tower [N, Beg, Aux, End]

For a non-recursive sort to the tower of Hanoi problem for end is which is obtained by discarding the recursive sort stack STN, STBeg, STAux, STend, STAdd with corresponding respectively to the variables, N, Beg, Aux, End, Add

3. Set Top = Null

4. If $N = 1$, then

- a. Write Beg $\rightarrow$ End
- b. go to steps
- [End of 4th step]

5. Translation of call Tower [N-1, Beg, Aux, End]

(a) Set Push current values onto stack

6. i. Set Top = Top + 1

ii. Set STN [Top] = N, STBeg [Top] = Beg

STAux [Top] = Aux, STEnd [Top] = end, STADD [Top] = Add

(b) [Reset Parameters]

Set N = N-1, Beg = Beg, temp = Aux

Aux = end, End = temp

(c) go to step 1

8. Write Beg $\rightarrow$ End

9. Translation of call Tower [N-1, Aux, Beg, End]

a. [Push current values and new return

i. Set Top = Top + 1

ii. Set STN [Top] = N, STN Beg [Top] = Beg

STAux [Top] = Aux, ST End [Top] = End

STADD [Top] = 5

iii. Set Top = Top - 1

b. [Reset Parameters]

Set $N = N-1$, temp = Beg, Beg = Aux, Aux = temp, End = End
 c. go to step 1

5. [Translation of Return]

a. If Top = Null, Return
 b. [Return Top values]

c. f Set $N = STN(Top)$, Beg = $STN(Top)$, Aux = $STN(Top)$
 if End = $STEnd(Top)$, Add = $STADD(Top)$

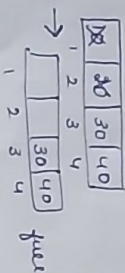
if Top = Top - 1

c. go to keep add

Unit-IV

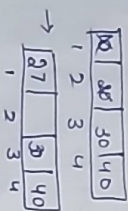
Queues

Linear queue:



input: 10
 " 20
 " 30
 " 40
 output: 10
 delete: 20
 insert: 27

Circular queue



Insert (B, N)

[Queue is full?]

If rear = N,

front = "Queue is full"

Return

[End of if sta]

If front = rear = 0 [Queue is empty]

front = rear = 1

else

rear = rear + 1

8 [array] = item

Exit

⇒ Queue (B, N)

1 [Queue is empty?]

2 If front = rear = 0

then front = "Queue is empty"

return

3 Set item = B[front]

4 If front = rear, then

front = rear = 0

else

front = front + 1

5 Exit

⇒ Insertion into circular queue

Q Insert [Queue, N, Front, rear, item]

1 [Queue already filled?]

If front = 1 and rear = N or

if front = rear + 1, then

return overflow and return

2 [Find new value of rear]

If front = null, then [Queue initially empty]

Set front = rear = 1

else if rear = N, then

rear = 1

else

rear = rear + 1

[End of if sta]

3 Set Queue[rear] = item [This inserts new element]

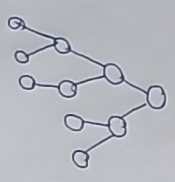
4 Return

- left sub-tree
- right "
- ancestor / grandparent
- Terminal nodes / leaf nodes / External nodes (no child)
- Non terminal nodes | Internal nodes
- path (consecutive set of edges)
- edge (line connecting 2 nodes)

Similar trees
same structure as well as node values

- depth / height (level + 1)
- height level (root = 0 and other successors)
- number of nodes at level $n = 2^n$

→ Complete binary tree
no of nodes = max no of nodes (except possibly the last)



max no of nodes in a tree of level n is $\log_2 n + 1$
depth of a complete tree with n nodes is $\lceil \log_2 n + 1 \rceil$
eg: $n = 7$
 $\lceil \log_2 7 + 1 \rceil = \lceil 2.8 + 1 \rceil = \lceil 3.8 \rceil = 4$

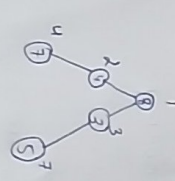
→ Representation of trees in memory.

Array representation of trees

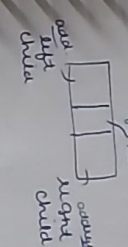
root = 1
parent = n
left child = $2 * n$
right child = $(2 * n + 1)$

1	2	3	4	5	6	7	8
	8	4	3	7			

Not efficient

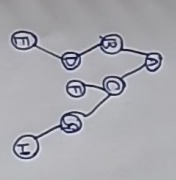


Linked Representation



Binary Tree Traversal

- Pre order - Root, left, right
- In order - left, root, right
- Post order - left, right, root



→ In order:
BEDAFCGH
→ Pre order:
ABDECFGH
→ Post order:
EDBFHACG

Non-recursive algorithms of tree traversals

① Pre order

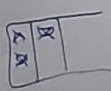
- root
- visit push
- move to left, if left is not null else pop

② In order

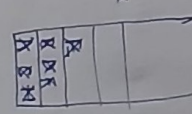
- push until left is not null
- pop & and process if left is not null
- move to right child if it is not null

③ Post order

- push root on to stack
- push right "
- push left "
- move to left child if it is not null
- else pop from stack
- if true node, process and pop
- if -ve, move to step 1



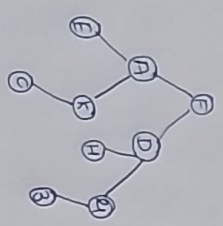
ABDECFGH



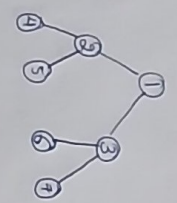
GEDBFCGH

EDBFCGH

① Inorder - E A C K F H D B G
Pre order - F A F K C D H G B



② In 4 2 5 1 6 3 7
Post 4 5 2 6 7 3 1



⇒ Header Nodes
⇒ Threading

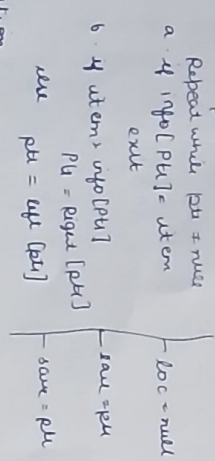
⇒ Binary Search Tree
avg. running time - $f(n) = O(\log_2 n)$

② Sorted linear array
 $f(n) = O(\log_2 n)$

→ insert and delete operations are expensive

③ Linked list - insertion & deletion easy
Search → expensive, $f(n) = O(n)$
Linear search

⇒ Searching in a binary search tree:



⇒ Insertion

1. If $loc \neq null$, exit
2. create a new node
3. if $[new].item, left [save] = right [save]; null$
4. if $item < left [save]$
 $left [save] = new$
else
 $right [save] = new$

⇒ Deletion

- Three cases:
1. no child
 2. one child
 3. Two child

Search loc / pt

if $left[loc] = null$ and $right[loc] = null$

if $item < left[pt]$

$left[pt] = null$

else
 $right[pt] = null$

else

if $loc == null$

1. $pt = right[loc]$

$save = loc$

Repeat while $left[pt] \neq null$

$save = pt, pt = left[pt]$ [end of null]

$ptsave = save$
 $suc = pt$

② If $ptsave \neq null$

$right[save] = null$

$child = right[save]$

else
 $child = null$

③ If $ptsave = null$

$if suc = left[ptsave]$

$left[ptsave] = child$

else

$right[ptsave] = child$

if $ptsave \neq null$

$if loc = left[ptsave]$

$right[ptsave] = suc$

else

$right[ptsave] = suc$

else

$Root = suc$

AVL Search Tree

Insertion

LL

RR

RL

LR (RR followed by LL)

→ Rotation in AVL Search Tree

Let A be the root node. On the path from node (X → deleted node)

X is root node with BF 2 or -2.

Rotations

L or R depending on rotation occurred in left or right subtree

Let A.

New depending on value of BF (B). B is root of left or right subtree

subtree of A.

Root imbalance is further classified as

RL, RR, LL, LR, LR, LR, LR, LR

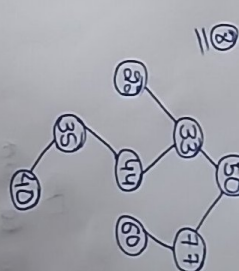
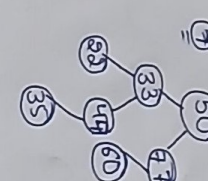
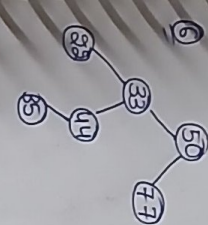
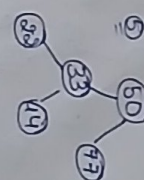
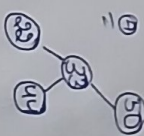
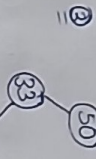
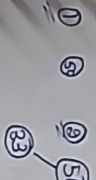
LL of Root is identical

LL of RL " " but different balance factor.

LR of RL " " "

Q. Construct the following nodes in order into empty

50, 33, 44, 22, 77, 35, 60, 40



Inorder

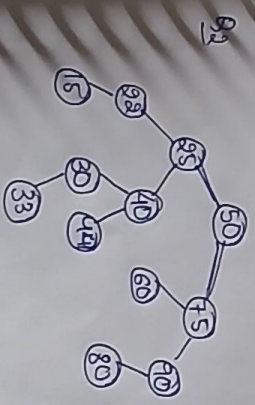
22 33 35 40 44 50 60 77

Pre order

50 33 22 44 35 40 77 60

Post order

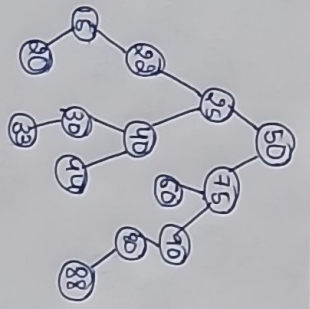
22 40 35 44 33 60 77 50



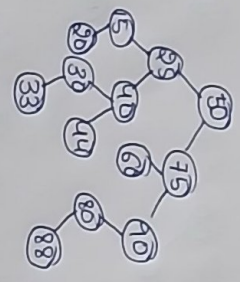
Insert 20
Insert 15
Insert 40
Insert 22
Insert 44
Insert 75

Handwritten signature

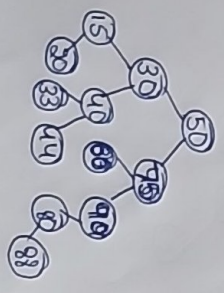
Handwritten signature



class 28



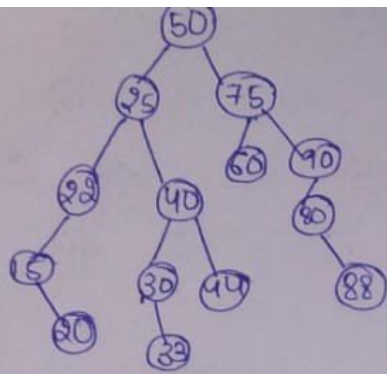
class 25



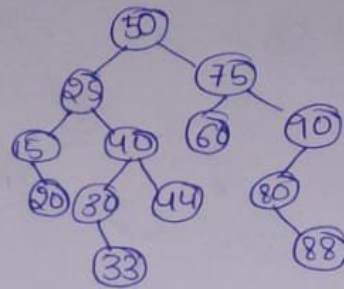
1011125

⇒ Heap Sort

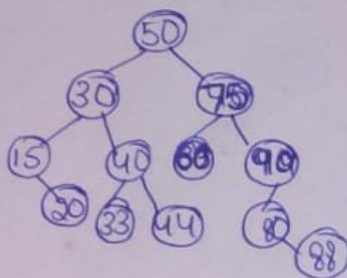
heap → data structure
 → complete binary tree
 → array representation
 Max heap
 Min heap



delete 22



delete 25



10/11/25

⇒ Heap Sort

- heap → data structure
- complete binary tree
- array representation

heap
Max heap Min heap

⇒ m-way search tree

To faster retrieval and manipulation of data, store in external memory i.e. storage devices such as disk etc. There is a need for some special data-structures. m-way search tree, B-tree, B+tree are ex. of D.S. which finds applications in problems such as file indexing. m-way search trees are generalised version of ^{B-tree} B.S.T.

height h is close to $\log_m(n+1)$ no. of elements in m-way tree of height h ranges from min to max $m^h - 1$.

→ Insertion

→ Deletion

B-Tree

A B-tree of order m , is m -way search tree in which

- ① root has atleast two child nodes and atmost m child nodes
- ② the internal nodes except the root have atleast $\lceil \frac{m}{2} \rceil$ child nodes and atmost m child nodes.

③

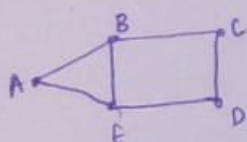
- ④ all leaf nodes are at same level.

→ Insertion in B-Tree

→ Deletion in B-Tree

GRAPHS:

→ Graphs (V, E) Set of edges
Set of vertices



$V = A, B, C, D, E$
 $E = AB, BC, CD, DE, AE, BE$

Graph		Tree
- cycle		- no cycle
		- no vertices
$\max \frac{n(n-1)}{2}$		$(n-1)$ edge

→ Representation of graph in memory

→ Sequential [with array]

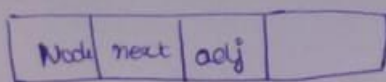
Adjacency Matrix $a_{ij} = \begin{cases} 1 & \text{if } v_i \text{ is adjacent to } v_j \text{ that is} \\ & \text{there is an edge } (v_i, v_j) \\ 0 & \text{otherwise} \end{cases}$

Path Matrix

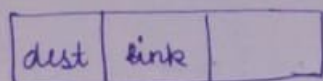
$p_{ij} = \begin{cases} 1 & \text{if there is path from } v_i \text{ to } v_j \\ 0 & \text{otherwise} \end{cases}$

→ Linked Representation

⊙ Node list



⊙ edge list

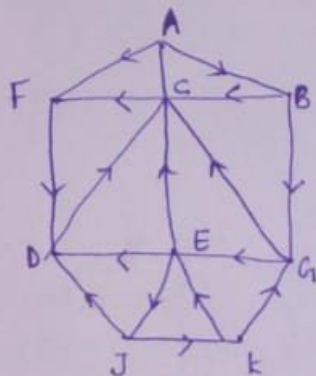


⇒ Traversing a graph

status=1 Ready state

status=2 waiting state

status=3 Processed state



A F, C, B

B C, G

C F

D C

E D, J, C

F D

G C, E

J D, K

K E, G

① Breadth first search - Queue

② Depth " " - Stack

- (iii) For a node $A_0, (K_1, A_1), (K_2, A_2), \dots, (K_{m-1}, A_{m-1})$, all key values in the subtree pointed to by A_i are less than the key K_{i+1} , $0 \leq i \leq m-2$, and all key values in the subtree pointed to by A_{m-1} are greater than K_{m-1} .
- (iv) Each of the subtrees $A_i, 0 \leq i \leq m-1$ are also m -way search trees.

An example of a 5-way search tree is shown in Fig. 7.45. Observe how each node has at most 4 child nodes and therefore has at most 4 keys contained in it. The rest of the properties of an m -way search tree can also be verified by the example.

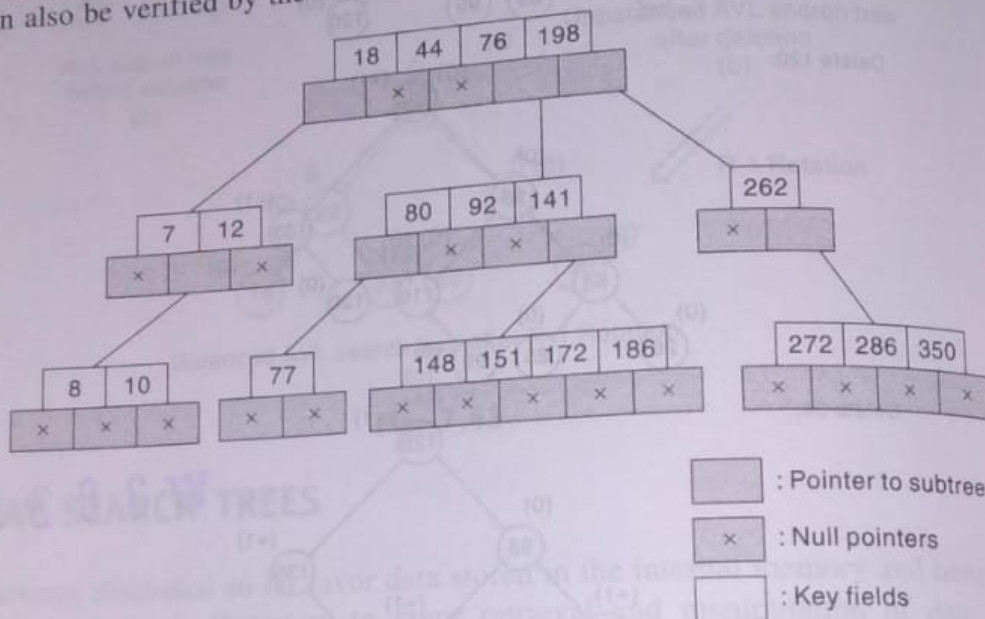


Fig. 7.45

7.14 SEARCHING, INSERTION AND DELETION IN AN m -WAY SEARCH TREE

Searching an m -Way Search Tree

Searching for a key in an m -way search tree is similar to that of binary search trees. To search for 77 in the 5-way search tree, shown in Fig. 7.45, we begin at the root and as $77 > 76 > 44 > 18$, we move to the fourth subtree. In the root node of the fourth subtree, $77 < 80$ and therefore we move to the first subtree of the node. Since 77 is available in the only node of this subtree, we claim 77 was successfully searched.

To search for 13, we begin at the root and as $13 < 18$, we move down to the first subtree. Again, since $13 > 12$ we proceed to move down the third subtree of the node but fall off the tree having encountered a null pointer. Therefore we claim the search for 13 was unsuccessful.

Insertion in an m -Way Search Tree

To insert a new element into an m -way search tree we proceed in the same way as one would in

order to search for the element. To insert 6 into the 5-way search tree shown in Fig. 7.45, we proceed to search for 6 and find that we fall off the tree at the node [7, 12] with the first child node showing a null pointer. Since the node has only two keys and a 5-way search tree can accommodate up to 4 keys in a node, 6 is inserted into the node as [6, 7, 12]. But to insert 146, the node [148, 151, 172, 186] is already full, hence we open a new child node and insert 146 into it. Both these insertions have been illustrated in Fig. 7.46.

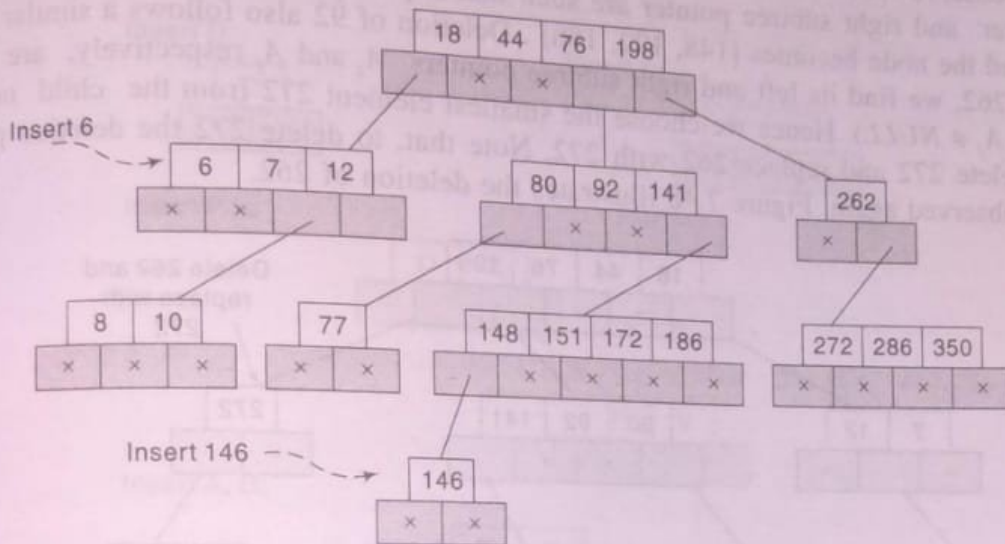


Fig. 7.46

Deletion in an m -Way Search Tree

Let K be a key to be deleted from the m -way search tree. To delete the key we proceed as one would to search for the key. Let the node accommodating the key be as illustrated in Fig. 7.47.

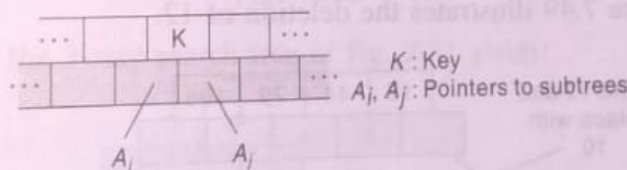


Fig. 7.47

If $(A_i = A_j = \text{NULL})$ then delete K .

If $(A_i \neq \text{NULL}, A_j = \text{NULL})$ then choose the largest of the key elements K' in the child node pointed to by A_i , delete the key K' and replace K by K' . Obviously deletion of K' may call for subsequent replacements and therefore deletions in a similar manner, to enable the key K' move up the tree.

If $(A_i = \text{NULL}, A_j \neq \text{NULL})$ then choose the smallest of the key elements K'' from the subtree pointed to by A_j , delete K'' and replace K by K'' . Again deletion of K'' may trigger subsequent replacements and deletions to enable K'' move up the tree.

If $(A_i \neq \text{NULL}, A_j \neq \text{NULL})$ then choose either the largest of the key elements K' in the subtree pointed to by A_i or the smallest of the key elements K'' from the subtree pointed to by A_j to replace K . As mentioned before, to move K' or K'' up the tree it may call for subsequent replacements and deletions.

We illustrate deletions on the 5-way search tree shown in Fig. 7.45. To delete 151, we search for 151 and observe that in the leaf node [148, 151, 172, 186] where it is present, both its left subtree pointer and right subtree pointer are such that $(A_i = A_j = \text{NULL})$. We therefore simply delete 151 and the node becomes [148, 172, 186]. Deletion of 92 also follows a similar process.

To delete 262, we find its left and right subtree pointers A_i and A_j respectively, are such that $(A_i = \text{NULL}, A_j \neq \text{NULL})$. Hence we choose the largest of the keys from the child node [272, 286, 300], delete 272 and replace 262 with 272. Note that, to delete 272 the deletion procedure needs to be observed again. Figure 7.48 illustrates the deletion of 262.

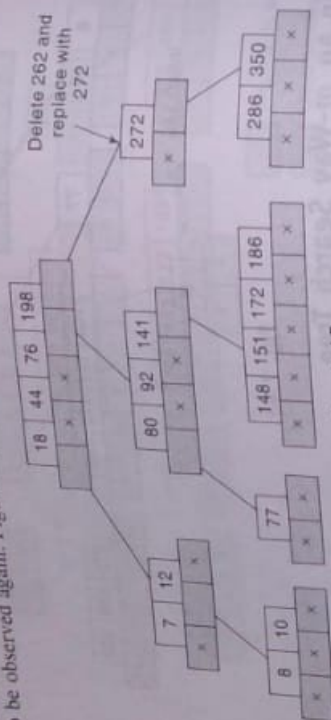


Fig. 7.48

To delete 12, we find the node [7, 12] accommodates 12 and the key satisfies $(A_i \neq \text{NULL}, A_j = \text{NULL})$. Hence we choose the largest of the keys from the node pointed to by A_i , viz., 10 and replace 12 with 10. Figure 7.49 illustrates the deletion of 12.

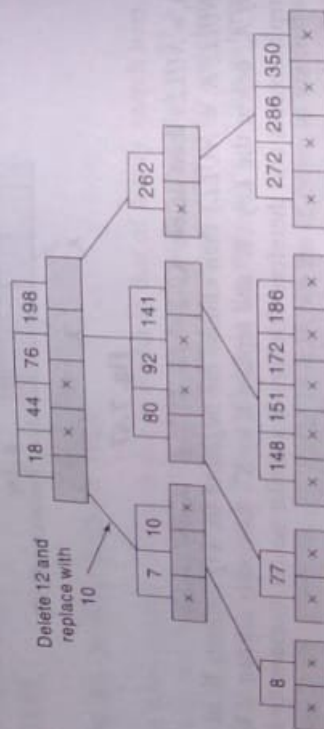


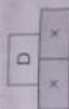
Fig. 7.49

Example 7.27

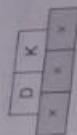
A 3-way search tree constructed out of an empty search tree with the following keys in the order shown, is illustrated in Fig. 7.50:

D, K, P, V, A, G

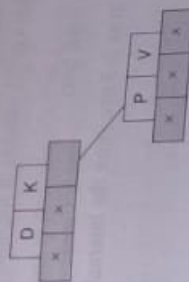
Insert D:



Insert K:



Insert P, V:



Insert A, G:

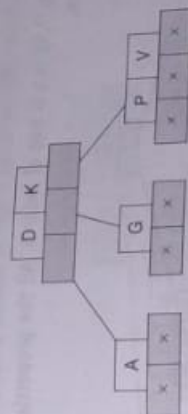


Fig. 7.50

Deletion of A and K in the 3-way search tree of Fig. 7.51 yields:

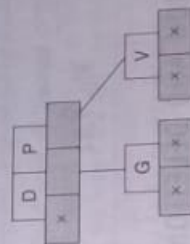


Fig. 7.51

7.15 B TREES

m -way search trees have the advantage of minimizing file accesses due to their restricted height. However it is essential that the height of the tree be kept as low as possible and therefore there

balanced m -way search trees. Such a balanced m -way search tree arises the need to maintain what is defined as a B tree.

Definition

A B tree of order m , if non empty, is an m -way search tree in which:

- (i) the root has at least two child nodes and at most m child nodes
 - (ii) the internal nodes except the root have at least $\lceil \frac{m}{2} \rceil$ child nodes and at most m child nodes
 - (iii) the number of keys in each internal node is one less than the number of child nodes of the node in a manner similar to that of B search trees.
 - (iv) all leaf nodes are on the same level
- A B-tree of order 3 is referred to as 2-3 tree since the internal nodes are of degree 2 or 3.

Example 7.28

The tree shown in Fig. 7.52 is a B tree of order 5. All the properties of the B tree can be verified on the tree.

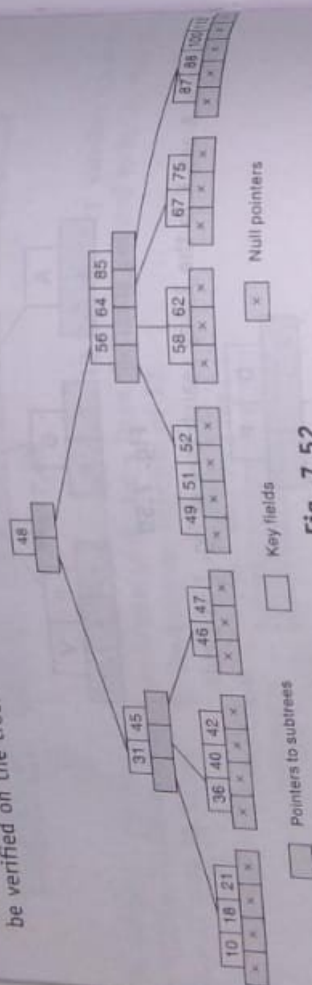


Fig. 7.52

7.16 SEARCHING, INSERTION AND DELETION IN A B-TREE

Searching a B-tree

Searching for a key in a B-tree is similar to the one on an m -way search tree. The number of accesses depends on the height h of the B-tree.

Insertion in a B-tree

The insertion of a key in a B-tree proceeds as if one were searching for the key in the tree. When the search terminates in a failure at a leaf node and tends to fall off the tree, the key is inserted according to the following procedure:

If the leaf node in which the key is to be inserted is not full, then the insertion is done in the node. A node is said to be full if it contains a maximum of $(m-1)$ keys, given the order of the B-tree to be m .

If the node were to be full, then insert the key in order into the existing set of keys in the node, split the node at its median into two nodes at the same level, pushing the median element up to the parent node. Note that the split nodes are only half full. Accommodate the median element up by rearrangement of the keys in the root node or the formation of a new root itself. This is a major observation pertaining to insertion in a B-tree is that, since the leaf nodes are all at the same level, unlike m -way search trees, the tree grows upwards.

Example 7.29

Consider the B-tree of order 5 shown in Fig. 7.53. Insert the elements 4, 5, 58, 6 in the order given.

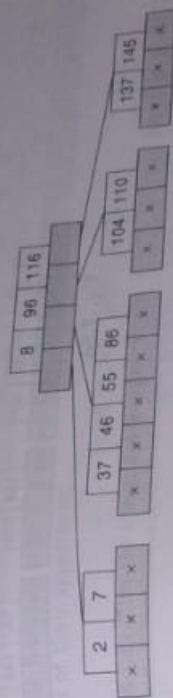


Fig. 7.53

The insertion of the given elements is shown in Fig. 7.54. Note how insertion of 4, 5 is easily done in the first child node of the root since the node is not full. But during the insertion of 58, the second child node of the root, where the insertion needs to be done is full. Now we insert the key into the list of keys existing in the node in the sorted order and split the node into two nodes at the same level at its median, viz., 55. Push the key 55 up to accommodate it in the parent node which is not full. Thus the key 58 is inserted.

The insertion of 6 is interesting. The first child node of the root where its place is due, is full. Now the node splits at the median pushing 5 up. But the parent node viz., the root, is also full. This in turn forces a split in the root node resulting in the creation of a new root which is 55.

It needs to be highlighted that all the keys in the node should be ordered every time there is a rearrangement of keys in a node.

Insert 4, 5



Insert 58:



Insert 6:

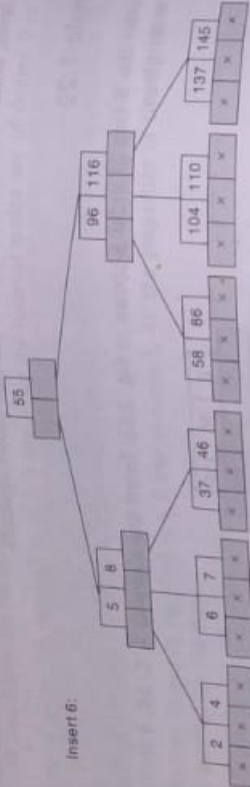


Fig. 7.54

Deletion in a B-tree

While deleting a key it is desirable that the keys in the leaf node are removed. However when a case arises forcing a key that is in an internal node to be deleted, then we promote a successor or a predecessor of the key to be deleted, to occupy the position of the deleted key and such a key is bound to occur in a leaf node (How?).

While removing a key from a leaf node, if the node contains more than the minimum number of elements, then the key can be easily removed. However, if the leaf node contains just the minimum number of elements, then scout for an element from either the left sibling node or right sibling node to fill the vacancy. If the left sibling node has more than the minimum number of keys, pull the largest key up into the parent node and move down the intervening number of keys, pull the smallest key up into the parent node and move down the intervening entry from the parent node to the leaf node where the key is to be deleted. Otherwise, pull the smallest key of the right sibling node to the parent node and move down the intervening parent element to the leaf node.

If both the sibling nodes have only minimum number of entries, then create a new leaf node out of the two leaf nodes and the intervening element of the parent node, ensuring that the total number does not exceed the maximum limit for a node. If while borrowing the intervening element from the parent node, it leaves the number of keys in the parent node to be below the minimum number, then we propagate the process upwards ultimately resulting in a reduction of height of the B-tree.

Example 7.30

Deletion of keys 95, 226, 221 and 70 on a given B-tree of order 5 is shown in Fig. 7.55. The deletion of key 95 is simple and straight since the leaf node has more than the minimum number of elements. To delete 226, the internal node has only the minimum number of elements and hence borrows the immediate successor viz., 300 from the leaf node which has more than the minimum number of elements. Deletion of 221 calls for the hauling of key 440 to the parent node and pulling down of 300 to take the place of the deleted entry in the leaf. Lastly the deletion of 70 is a little more involved in process. Since none of the adjacent leaf nodes can afford lending a key, two of the leaf nodes are combined with the intervening element from the parent to form a new leaf node, viz., [32, 44, 65, 81] leaving 86 alone in the parent node. This is not possible since the parent node is now running low on its minimum number of elements. Hence we once again proceed to combine the adjacent sibling nodes of the specified parent node with a median element of the parent which is the root. This yields the node [86, 110, 120, 440] which is the new root. Observe the reduction in height of the B-tree.

Example 7.31

On the B-tree of order 3 shown in Fig. 7.56, perform the following operations in the order of their appearance:

Insert 75, 57 Delete 35, 65

The B-tree after the operations is shown in Fig. 7.57

1.17 HEAP; HEAPSORT

This section discusses another tree structure, called a *heap*. The heap is used in an elegant sorting algorithm called *heapsort*. Although sorting will be treated mainly in Chapter 9, we give the heapsort algorithm here and compare its complexity with that of the bubble sort and quicksort algorithms, which were discussed, respectively, in Chaps. 4 and 6.

Suppose H is a complete binary tree with n elements. (Unless otherwise stated, we assume that H is maintained in memory by a linear array *TREE* using the sequential representation of H , not a linked representation.) Then H is called a *heap*, or a *maxheap*, if each node N of H has the following property: *The value at N is greater than or equal to the value at each of the children of N .* Accordingly, the value at N is greater than or equal to the value at any of the descendants of N . A *minheap* is defined analogously: The value at N is less than or equal to the value at any of the children of N .)